

Digit Rearrangement and Palindrome in C

Algorithms Test Cases Program and Execution

Name Reyansh Pandit

Batch 16

SAP ID 590033602

1 Problem and interpretation

This menu-driven program performs four operations using separate parameterless void functions. Each function reads and validates its own positive integer, processes it, and displays the result. The smallest arrangement never begins with zero; the smallest nonzero digit is placed first, followed by all zeroes and the remaining digits in ascending order.

2 Algorithms

Part	Algorithm
Main menu	Display choices 1 to 5, read choice, and use switch-case to make a simple call to the selected void function. Repeat with do-while until choice 5.
Largest arrangement	Read and validate the number. For digits 9 down to 0, count each digit's occurrences by scanning the number. Append that digit to result the counted number of times. Display result.
Smallest arrangement	Read and validate the number. Find and append the smallest nonzero digit once. Append all zeroes. Then append the remaining digits from smallest to largest, preserving occurrences. Display result.
Swap first and last	Read and validate the number. Find the highest power-of-10 divisor. Extract first, last, and middle parts. Calculate last * divisor + middle * 10 + first. Display result.
Palindrome	Read and validate the number. Reverse it by repeatedly extracting the last digit. Compare the reverse with the original and display the appropriate message.

3 Test case table

Category	Input	Expected	Observed	Status
Invalid input	choice 1 then -5, 0, 321	Reject -5 and 0; accept 321	As expected	Pass
Largest normal	31020	32100	32100	Pass
Largest repeats	50552	55520	55520	Pass
Largest one digit	7	7	7	Pass
Smallest zeroes	31020	10023	10023	Pass
Smallest boundary	1000	1000	1000	Pass
Smallest no zero	5312	1235	1235	Pass
Swap normal	12345	52341	52341	Pass
Swap ending zero	1230	231	231	Pass
Swap one digit	7	7	7	Pass
Palindrome true	12321	Palindrome	Palindrome	Pass
Palindrome false	123	Not palindrome	Not palindrome	Pass
Invalid menu	choice 6	Invalid choice	Invalid choice	Pass

4 C program

```
#include <stdio.h>

void generate_largest(void);
void generate_smallest(void);
void swap_first_last(void);
void check_palindrome(void);

void generate_largest(void)
{
    int number, digit, temp, count, result = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    for (digit = 9; digit >= 0; digit--) {
        count = 0;
        for (temp = number; temp != 0; temp /= 10)
            if (temp % 10 == digit)
                count++;
        while (count > 0) {
            result = result * 10 + digit;
            count--;
        }
    }

    printf("Largest possible number = %d\n", result);
}

void generate_smallest(void)
{
    int number, digit, temp, count, first_nonzero = 0, result = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    for (digit = 1; digit <= 9 && first_nonzero == 0; digit++)
        for (temp = number; temp != 0; temp /= 10)
            if (temp % 10 == digit) {
                first_nonzero = digit;
                break;
            }

    result = first_nonzero;

    count = 0;
    for (temp = number; temp != 0; temp /= 10)
        if (temp % 10 == 0)
            count++;
    while (count > 0) {
        result *= 10;
        count--;
    }

    for (digit = first_nonzero; digit <= 9; digit++) {
        count = 0;
        for (temp = number; temp != 0; temp /= 10)
            if (temp % 10 == digit)
                count++;
        if (digit == first_nonzero)
            count--;
        while (count > 0) {
            result = result * 10 + digit;
            count--;
        }
    }

    printf("Smallest possible number = %d\n", result);
}

void swap_first_last(void)
{
    int number, divisor = 1, first, last, middle, result;
```

```

do {
    printf("Enter a positive integer: ");
    scanf("%d", &number);
    if (number <= 0)
        printf("Invalid input. Enter a positive integer.\n");
} while (number <= 0);

while (number / divisor >= 10)
    divisor *= 10;

first = number / divisor;
last = number % 10;
middle = (number % divisor) / 10;
result = last * divisor + middle * 10 + first;

printf("Number after swapping first and last digits = %d\n", result);
}

void check_palindrome(void)
{
    int number, original, temp, reverse = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    original = number;
    for (temp = number; temp != 0; temp /= 10)
        reverse = reverse * 10 + temp % 10;

    if (reverse == original)
        printf("%d is a palindrome number.\n", original);
    else
        printf("%d is not a palindrome number.\n", original);
}

int main(void)
{
    int choice;

    do {
        printf("\n1. Generate the largest possible number\n");
        printf("2. Generate the smallest possible number\n");
        printf("3. Swap the first and last digits\n");
        printf("4. Check whether the number is a palindrome\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: generate_largest(); break;
            case 2: generate_smallest(); break;
            case 3: swap_first_last(); break;
            case 4: check_palindrome(); break;
            case 5: printf("Program terminated.\n"); break;
            default: printf("Invalid choice. Select 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}

```

5 Compilation

```
gcc -std=c11 -Wall -Wextra -pedantic digit_rearrangement.c -o digit_rearrangement
```

Compilation completed successfully with no warnings or errors.

6 Execution

```
(kali)kali)-[~/demo1]
$ gcc -std=c11 -Wall -Wextra -pedantic digit_rearrangement.c -o digit_rearrangement
(kali)kali)-[~/demo1]
$ ./digit_rearrangement

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap the first and last digits
4. Check whether the number is a palindrome
5. Exit
Enter your choice: 1
Enter a positive integer: 31020
Largest possible number = 32100

Enter your choice: 2
Enter a positive integer: 31020
Smallest possible number = 10023

Enter your choice: 3
Enter a positive integer: 12345
Number after swapping first and last digits = 52341

Enter your choice: 4
Enter a positive integer: 12321
12321 is a palindrome number.

Enter your choice: 5
Program terminated.
```

Figure 1 Compilation and execution of all four operations

7 Test result

The program was compiled and executed against all prepared valid, invalid, boundary, repeated-digit, zero-containing, and palindrome cases. Every observed result matched its expected result.

8 Conclusion

The program meets the switch-case and parameterless void-function requirements. It correctly generates the largest and smallest digit arrangements, swaps the first and last digits, and tests whether a number is a palindrome.